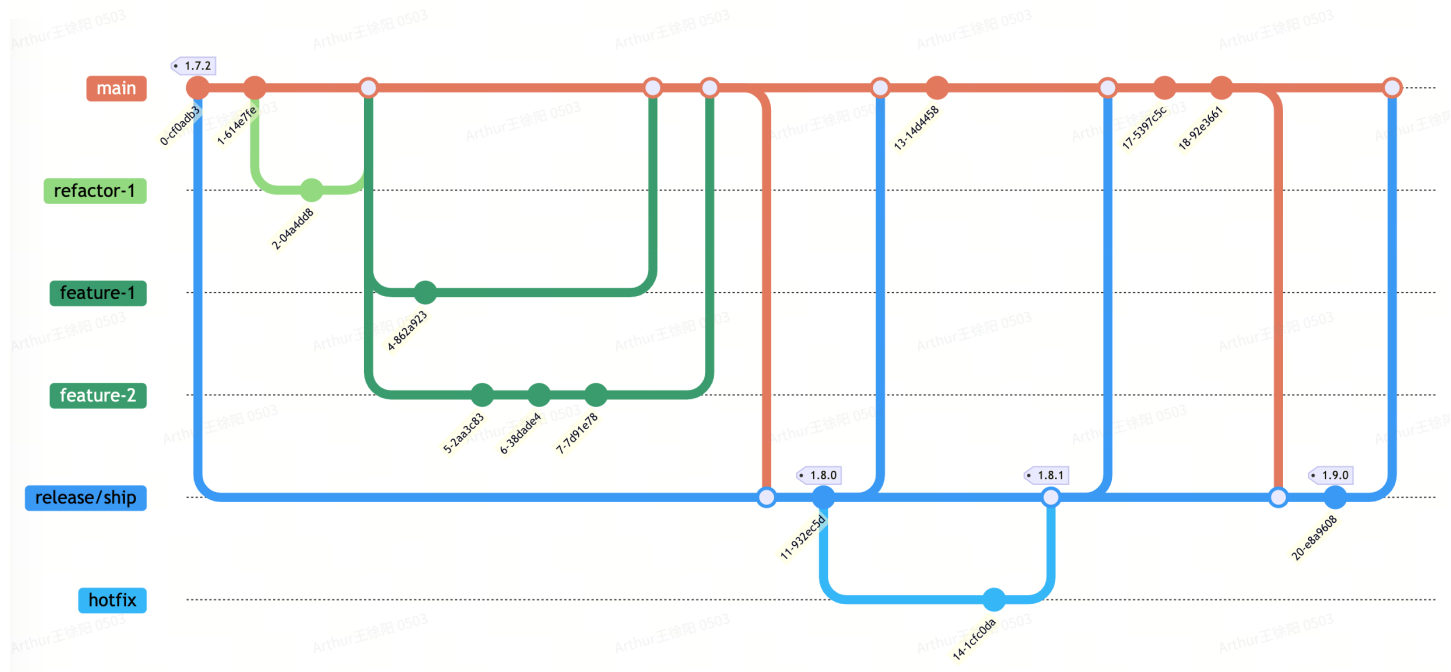


iOS Git Convention

1. 分支模型

如下图所示，稍作解释

- 核心思想基于 **main** 分支工作。
- 日常工作，对所有正在开展需求迭代都有用的改动（如图 **refactor-1**），从 **main** 中分出合入 **main**。
- 正常迭代需求开 **feature** 分支开展（如图 **feature-1**, **feature-2**），完成后合入 **main**。feature 分支在合并前，可以不断的 rebase **main** 来获取最新代码。
- 在需要发版时，在 **release** 分支 merge **main** 分支，在此进行 QA、bug fix 后发版，审核通过后台回 **main**。在 release 打上 tag 以做归档。
- 临时 hotfix 版本（如图 hotfix），从 release 最近的线上版本拉出分支，QA、bugfix 后发版，审核通过后台回 **release** 打 tag 存档。并 merge 回 **main**。



图中另外需要解释的是1.7.2那个commit，也就是左边那条蓝线，是我们当前工作方式的终点。

2. Commit message规范

原则上遵循github conventional commit message，使用英文，尽量准确描述scope

<https://gist.github.com/qoomon/5dfcdf8eec66a051ecd85625518cfd13>

[gist.github.com](https://gist.github.com/qoomon/5dfcdf8eec66a051ecd85625518cfd13)

建议先仔细阅读github的convention，非常细致。（比如细致到首字母不要大小写，最后不要加dot等等）

大致是

```
git
1  type(scope): message
2  description
```

- Type

- 常用

- 新功能 `feat(scope): xxx`
 - 修复 `fix(scope): xxx`
 - 杂事比如更新图片 `chore`
 - 重构 `refactor`

- 其他

- 改代码格式 `style`
 - 测试代码 `test`

- Scope

- 要加上，要加上，要和以前写相关代码的同事使用的scope保持一致

- Message

- 用英文
 - 客观描述改动的内容（而不是心路历程）

- 注意点

- 避免重复的无意义的commit message，一个commit做一件事，如果是反复修改相同的逻辑，尽可能用 `git commit --amend`，除非是已经push的协作分支。
 - 可以秉持一个原则。Commit message是写给其他人看的，不是写给自己看的，要让其他人明白这个commit干了什么。

3. Merge Request实践

1. 在你的feature分支初具规模的时候，就draft一个merge request，方便团队成员
 - a. 了解正在开展的工作
 - b. 有机会更早进行review，发现实现方案的大方向问题
2. 谁的MR谁负责，谁merge。reviewer不做「能approve就能merge」的预设。MR作者确认可以merge前不应当假设review过就可以merge，依然需要等待作者自己merge。
3. MR必须至少有一人review后才能merge（虽然gitlab免费版没有做此限制，但我们在流程上规范他）。
4. 谁着急谁push。不做「有MR就有人来review」的预设。谁着急谁催其他人review。
5. MR作者需要认真对待review的comment，作出相应的修改和解释。

4. 推荐的实践

1. 维护自己分支的时候，尽可能使用rebase更新来自其他分支的代码。
 - a. 比如你可以 `git pull --rebase origin main` 来获取主分支其他改动。或者确保本地main是最新的情况下，直接 `git rebase main`
 - b. rebase负担非常重的时候，意味着你的分支diverge太多/太久了，放弃rebase，直接merge main。注意一旦merge后，这个本地分支就不要再rebase了，应该用merge。
2. 提交MR前整理自己的改动
你可以 `git rebase -i` 来细致的fixup/squash你的改动，确保一个commit只做一件事
你也可以直接squash你的MR（不建议特别大的需求squash成一个commit）
3. 本地尽可能使用fast-forward merge，gitlab MR保持merge commit

以上规范遇到疑问的时候可以提问  Arthur王徐阳

附录

上面git flow图的mermaid源码

代码块

```
1  %[%{init: { 'logLevel': 'debug', 'theme': 'default', 'themeVariables': {
2      'git0': '#E68369',
3      'git1': '#9DDE8B',
4      'git2': '#40A578',
5      'git3': '#40A578',
6      'git4': '#3FA2F6',
```

```
7      'git5': '#3ABEF9',
8      'git6': '#9CDBA6',
9      'git7': '#ECCEAE'
10    } } }%%
11  gitGraph
12      commit tag: "1.7.2"
13      branch release/ship order: 3
14      checkout main
15      commit
16      branch refactor-1 order: 0
17      commit
18      checkout main
19      merge refactor-1
20      branch feature-2 order: 2
21      branch feature-1
22      commit
23      checkout feature-2
24      commit
25      commit
26      commit
27      checkout main
28      merge feature-1
29      merge feature-2
30      checkout release/ship
31      merge main
32      commit tag: "1.8.0" %% i.e. bump version
33      checkout main
34      merge release/ship
35      commit
36      checkout release/ship
37      branch hotfix order: 3
38      commit
39      checkout release/ship
40      merge hotfix tag: "1.8.1"
41      checkout main
42      merge release/ship
43      checkout main
44      commit
45      commit
46      checkout release/ship
47      merge main
48      commit tag: "1.9.0"
49      checkout main
50      merge release/ship
51
```